Project Assignment 2

Compiler Design Lab
Prof. Dr. Sebastian Hack
Fabian Ritter, B.Sc.

This project assignment consists of two parts: checking the input for syntactical correctness and generating (and printing) the abstract syntax tree. You may tackle them one after the other or implement them at the same time.

Project Task C Parser

Implement a parser for C[4]:

- For expressions (§6.5) we handle *identifier*, *constant*, *string-literal*, parenthesized expression, `[]`, function call, `.`, `->`, `sizeof`, `&` (unary), $\star$ (unary), $-$ (unary), `!`, $\star$ (binary), $+$ (binary), $-$ (binary), $<$, $==$, `!=`, `&&`, `||`, `?:` and $=$. For all other expressions only the chain productions ($A \to B$) are used.
- At declarations (§6.7) we only consider *init-declarator-list* with at most one *init-declarator* without *initializer*. The only *declaration-specifiers* and *specifier-qualifier-list* is *type-specifier*. *type-specifier* is restricted to `void`, `char`, `int` and *struct-or-union-specifier*. The latter is only `struct` without *type-qualifier*s and bit fields. *declarator* and *direct-declarator* are *pointer* (without *type-qualifier-list*), *identifier*, parenthesized declarator and function declarator with *parameter-type-list*. *parameter-type-list* is only *parameter-list* without ellipses (`...`). All productions for *parameter-declaration* are considered. The productions for *abstract-declarator* and *direct-abstract-declarator* are restricted accordingly.
- The considered *statement*s (§6.8) are *labeled-statement* with an identifier, *compound-statement*, *expression-statement* (both expression and null statements), *selection-statement* with `if` and `if-else`, *iteration-statement* with `while` and every *jump-statement*.
- The root are external definitions (§6.9), which are handled fully except for *declaration-list* in *function-definition*.

For parsing your compiler will be invoked with `c4 --parse test.c`. The parser must reject all words that are not derivable from the full grammar. The parser must accept all correct programs according to the restricted grammar. As error locations, choose the location of (the start of) the first token that makes the token sequence not a word of the grammar.[1] If the error is caused by a missing token at the end of the token sequence, report the location of the end of the file, i.e. the location after the last character of the file.

Remarks and hints:

- The grammar as given is not suitable for LL parsing in some places. For these places, adjust the grammar accordingly and/or use precedence climbing (which will be discussed in the lectures soon).
- Don't repeat yourself! Factorize common operations into helper functions.
- The grammar is ambiguous for *selection-statement*s. How is this ambiguity resolved in the language standard? How can this be treated in the implemenation of the parser?
- How to implement the k-lookahead capability in your lexer/parser?
- What classes and class hierarchy for AST nodes do you need, e.g. `Expression`, `BinaryExpression`?
- You may choose to delay some checks required to ensure that the input is a word of the grammar to the later semantic analysis stage. Once you implement it, the semantic analysis will run together with your parser in the tests.

[1] While this is not necessary the most helpful error location for developers using the compiler, it is better for automated testing.

Project Task D Pretty Printing

Up to now your compiler can already lex and parse input. In this project phase you will add a pretty printer to your compiler, i.e., functionality to generate nicely formatted source code from the parse tree. Therefore, you have to construct an abstract syntax tree (AST). Each AST node should be able to dump itself.

For a syntactically correct input program, `c4 --print-ast [file]` must print the output to stdout. This new switch must *augment* the `c4 --parse` functionality, i.e., its acceptance and rejection behavior as well as the respective return codes.

Download the example file from the link on the website. Invoking `c4 --print-ast ex05_pretty.c` should emit a char-exact copy of the source file. You can use the tool `diff` to compare your output to the reference file. Once again: *Your output must exactly match the reference output*.

You can use the pretty printer to debug your compiler. For example, the expression `a * b + d + e` should be dumped as `(((a * b) + d) + e)`. If you see a different output, your compiler handles associativity/operator precedence incorrectly.

In general, follow the reference file for placing newlines, spacing and so forth. The example mostly follows K&R style. Additionally, consider the following guidelines for your pretty printer:

- Print everything in the original order of the source file.
- Use a tab character (`'\t'`) for one level of indentation; do not use spaces.
- Do not wrap long lines.
- Do not print digraphs. Print the regular punctuator instead.
- Each (sub-)*expression* and each (sub-)*declarator/abstract-declarator* must be fully parenthesized, except for atoms consisting of at most one token like `0`, `'c'`, `"abc"`, *identifier*s and so forth. In particular, original parentheses are discarded. Do not re-associate any subexpressions.
- Do not hesitate to ask questions in the Forum.
- The soft deadline for this milestone is 2022-12-09.
- Keep it simple!